

Capitolo 2

Background

Questo capitolo introduce i concetti necessari per leggere i Capitoli 3 e 4 senza assumere familiarità pregressa con i campi coinvolti. Si parte dalle reti neurali e dal loro addestramento (§2.1), per poi presentare il reinforcement learning, dal paradigma agente-ambiente alle estensioni del DQN usate in questo lavoro (§2.2), le reti neurali ricorrenti (§2.3) e le reti neurali a grafo, incluse le formulazioni confrontate sperimentalmente nel Capitolo 4 (§2.4). Il capitolo si chiude presentando il controllo semaforico come problema applicativo, la letteratura che lo affronta e ciò che resta aperto (§2.5).

2.1 Reti Neurali

Una **rete neurale** è una funzione parametrica che trasforma un input in un output attraverso una successione di funzioni matematiche, i cui parametri, chiamati pesi, vengono appresi dai dati invece di essere codificati manualmente. L'unità base è il *neurone artificiale* o *perceptrone*, mostrato in Figura 2.1: esso riceve un vettore di input $x = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, ne calcola una combinazione lineare pesata $z = w^\top x + b = \sum_{i=1}^n w_i x_i + b$ (con $w = (w_1, w_2, \dots, w_n) \in \mathbb{R}^n$ il vettore di pesi e $b \in \mathbb{R}$ un termine costante detto *bias*), e applica a z una funzione non lineare σ detta *funzione di attivazione*, producendo l'output $\hat{y} = \sigma(z)$.

Attivazioni comuni, mostrate in Figura 2.2, sono la sigmoide $\sigma(z) = 1/(1 + e^{-z})$, che comprime z in $(0, 1)$, la tangente iperbolica $\tanh(z) = (e^z - e^{-z})/(e^z + e^{-z})$, che lo comprime invece in $(-1, 1)$, e la ReLU (Rectified Linear Unit) $\max(0, z)$, che si limita ad azzerare i valori negativi. La non linearità è essenziale: senza di essa, comporre più trasformazioni lineari produrrebbe comunque una singola trasformazione lineare, non riuscendo ad apprendere delle relazioni complesse e non lineari presenti nei dati.

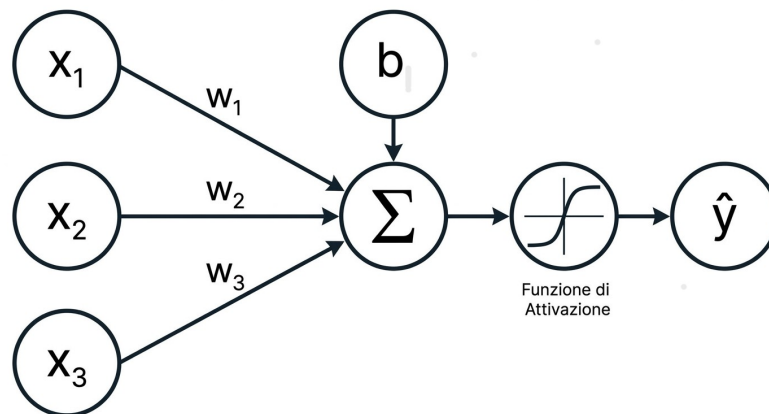


Figura 2.1: Il perceptrone: gli input x_1, x_2, x_3 vengono combinati con i pesi w_1, w_2, w_3 e il bias b , la somma z attraversa la funzione di attivazione, producendo l'output $\hat{y}$.

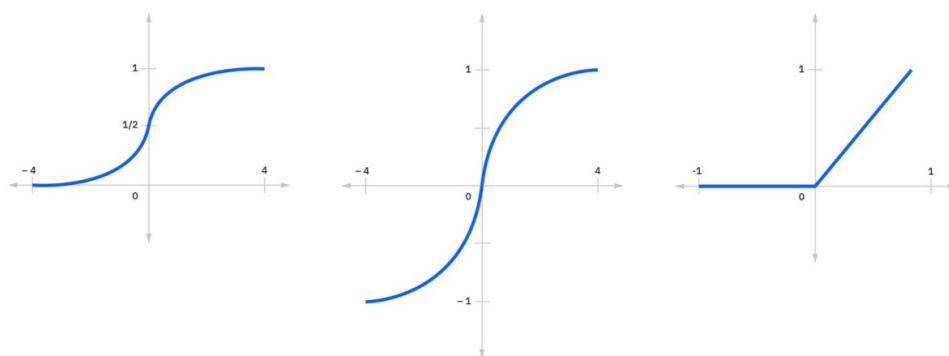


Figura 2.2: Le tre funzioni di attivazione discusse: sigmoide, tangente iperbolica e ReLU.

Una rete neurale *feedforward* o *multi-layer perceptron* (MLP) dispone i neuroni in **strati** (*layer*) successivi: l'output di ogni strato è l'input dello strato seguente, dall'*input layer* (i dati grezzi) attraverso uno o più *hidden layer* fino all'*output layer* finale, come in Figura 2.3. In forma matriciale, un singolo layer calcola

$$h^{(l+1)} = \sigma(W^{(l)}h^{(l)} + b^{(l)})$$

dove $h^{(l)}$ raccoglie gli output di tutti i neuroni dello strato l , $W^{(l)}$ è la matrice che impila i pesi di ciascun neurone di quello strato e $b^{(l)}$ è il vettore dei bias. Impilare più strati permette alla rete di comporre trasformazioni via via più astratte dell'input originale.

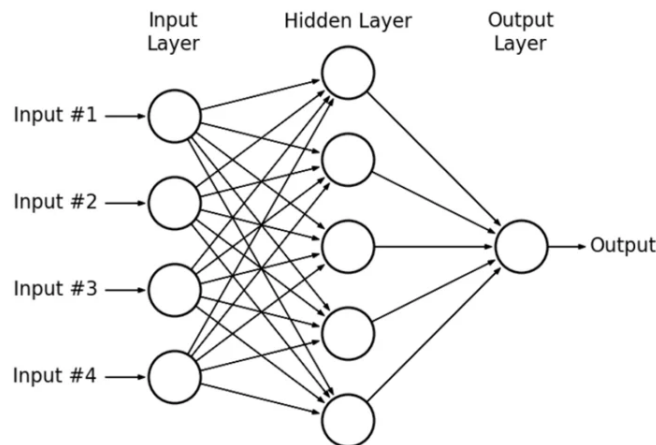


Figura 2.3: Una rete feedforward con un input layer a 4 nodi, un hidden layer a 5 nodi e un output layer a un nodo: ogni connessione porta con sé un peso appreso, omesso per chiarezza. Fonte: Wikimedia Commons.¹

Addestramento: discesa del gradiente e backpropagation. Una rete neurale viene inizializzata con pesi e bias casuali e viene addestrata iterativamente confrontando l'output prodotto con quello desiderato. Una funzione di loss $\mathcal{L}(\hat{y}, y)$ quantifica la distanza tra l'output prodotto $\hat{y}$ e quello desiderato y (che sia esso una etichetta di classificazione o un valore reale nel caso della regressione). L'addestramento consiste nel regolare ogni peso nella direzione che riduce $\mathcal{L}$, un'operazione nota come *discesa del gradiente*:

$$w \leftarrow w - \eta \frac{\partial \mathcal{L}}{\partial w},$$

dove η , il *tasso di apprendimento* o *learning rate*, controlla l'ampiezza di ogni aggiornamento. Calcolare la derivata parziale $\partial \mathcal{L} / \partial w$ per ogni peso della rete, dal

più vicino all'output al più lontano, richiede applicare ripetutamente la *chain rule* del calcolo differenziale attraverso tutti gli strati intermedi: l'algoritmo che esegue questo calcolo in modo efficiente, una sola passata dall'output verso l'input, è la *backpropagation* (Rumelhart et al., 1986). Nella pratica, il gradiente non è esatto ma si stima su un piccolo sottoinsieme casuale di dati (*mini-batch*) alla volta invece che sull'intero insieme di dati disponibile, una variante nota come discesa del gradiente stocastica, molto più economica computazionalmente a fronte di una stima del gradiente più rumorosa ma sufficiente a guidare comunque l'addestramento verso una soluzione utile.

Oltre la discesa del gradiente pura. La regola di aggiornamento vista sopra usa lo stesso tasso di apprendimento η per ogni peso della rete, un limite quando la superficie di loss ha curvatura molto diversa da un peso all'altro (alcune direzioni richiederebbero un passo grande, altre uno minimo): un ottimizzatore più sofisticato adatta l'ampiezza dell'aggiornamento parametro per parametro, sulla base della storia recente dei gradienti osservati. Il *momentum* (Qian, 1999) accumula una media mobile esponenziale dei gradienti passati e aggiorna i pesi in quella direzione media invece che nella sola direzione del gradiente corrente, smorzando le oscillazioni lungo le direzioni in cui il segno del gradiente cambia di continuo. **RMSprop** (Tieleman and Hinton, 2012) adotta un principio complementare: mantiene per ciascun peso una media mobile del quadrato dei gradienti recenti e divide l'aggiornamento per la sua radice, riducendo il passo sui pesi il cui gradiente è stato storicamente grande in valore assoluto e aumentandolo su quelli il cui gradiente è stato piccolo, così da rendere il progresso più uniforme tra pesi con scale molto diverse. **Adam** (Kingma and Ba, 2015), un altro ottimizzatore molto usato, combina invece entrambe le idee, media mobile del gradiente e del suo quadrato. Questo lavoro adotta **RMSprop**, coerentemente con il modello di riferimento (Wang et al., 2022), per l'addestramento delle proprie reti neurali.

Ogni rete usata in questo lavoro, dal DQN (§2.2) alla LSTM (§2.3) alle reti a grafo (§2.4), è in ultima analisi una particolare composizione di questi stessi due ingredienti: strati di neuroni con un'attivazione non lineare, addestrati per discesa del gradiente tramite backpropagation. Ciò che cambia da un'architettura all'altra è come gli strati sono collegati tra loro, non il principio di addestramento sottostante.

2.2 Reinforcement Learning

2.2.1 Il paradigma agente-ambiente

L'apprendimento per rinforzo o *reinforcement learning* (RL) è un paradigma di apprendimento automatico in cui un *agente* impara un certo comportamento interagendo ripetutamente con un *ambiente*, senza che gli siano forniti esempi di comportamento corretto. A ogni passo di tempo t , l'agente osserva lo stato corrente dell'ambiente, sceglie un'*azione* tra quelle disponibili, e riceve in cambio due informazioni: un nuovo stato (l'ambiente è cambiato, anche per effetto dell'azione scelta) e un numero scalare, la *ricompensa*, che indica quanto quell'azione sia stata desiderabile in quel momento. Il problema che il RL affronta è per l'appunto questo, cioè imparare per tentativi ripetuti quale comportamento massimizzi la somma delle ricompense ricevute nel tempo, non solo quella immediata. La Figura 2.4 riassume questo ciclo, utile come riferimento per la terminologia introdotta nel resto della sezione.

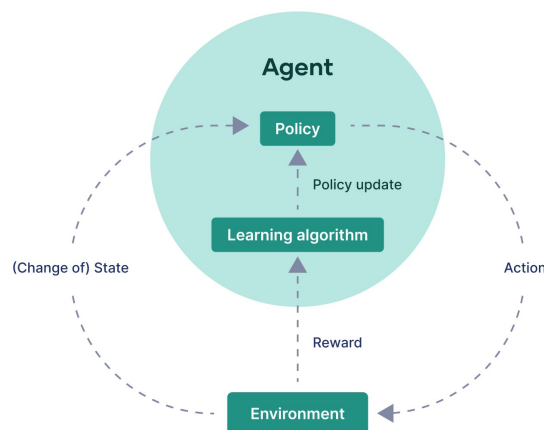


Figura 2.4: Il ciclo agente-ambiente: la policy sceglie un'azione, l'ambiente risponde con un nuovo stato e una ricompensa, e l'algoritmo di apprendimento usa quella ricompensa per aggiornare la policy. Fonte: adattata da un articolo Medium.²

Vale la pena distinguere questo paradigma da quello dell'*apprendimento supervisionato*. In quest'ultimo una rete neurale riceve coppie input-output correttamente associate e impara a riprodurre quella corrispondenza. Nel RL non esiste alcuna etichetta corretta da imitare: l'agente deve scoprire da solo, per tentativi, quale azione sia buona in ciascuno stato, guidato solo dalla ricompensa ricevuta in seguito alla azione che l'ha causata. È un problema strutturalmente più difficile, ma anche particolarmente adatto a situazioni, come il controllo semaforico, in cui nessuno conosce a priori quale sia la "decisione corretta" da etichettare.

2.2.2 Processo decisionale di Markov

La formalizzazione matematica standard di un problema di RL è il *processo decisionale di Markov* (MDP) (Sutton and Barto, 2018), definito dalla tupla

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

dove $\mathcal{S}$ è l'insieme di tutti gli stati possibili dell'ambiente, $\mathcal{A}$ l'insieme di tutte le azioni possibili, $P(s' | s, a)$ la probabilità di transitare nello stato s' avendo eseguito l'azione a nello stato s , $R(s, a)$ la ricompensa attesa per quella stessa coppia, e $\gamma \in [0, 1)$ il *fattore di sconto*, che pondera l'importanza delle ricompense future rispetto a quelle immediate: un γ vicino a 0 rende l'agente "miope", interessato quasi solo alla ricompensa immediata, un γ vicino a 1 lo rende attento anche a conseguenze lontane nel tempo. Questa formalizzazione si basa sull'assunzione di Markov, per la quale la probabilità di transizione $P(s' | s, a)$ dipende solo dallo stato corrente s e non dalla sequenza di stati che ha portato allo stesso.

Un agente segue una *politica* $\pi(a | s)$, una distribuzione di probabilità sulle azioni condizionata allo stato corrente; il suo obiettivo è massimizzare il *ritorno* atteso

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

ovvero la somma scontata delle ricompense future a partire dall'istante t . Per poter ragionare su quanto buona sia una politica prima ancora di eseguirla fino in fondo, si definiscono due funzioni valore: la funzione valore-stato $V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$ e la funzione valore-azione $Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a]$, che misurano rispettivamente quanto ritorno atteso si ottiene seguendo π da uno stato, o da una coppia stato-azione. Queste soddisfano l'*equazione di Bellman*, che esprime il valore di uno stato in funzione del valore atteso degli stati successivi:

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) [R(s, a) + \gamma V^\pi(s')] \quad (2.1)$$

Per una politica ottima π^* che massimizza $V^\pi(s)$ per ogni stato simultaneamente, vale l'analoga equazione di Bellman di ottimalità:

$$Q^*(s, a) = \mathbb{E}_{s' \sim P(\cdot | s, a)} \left[R(s, a) + \gamma \max_{a'} Q^*(s', a') \right] \quad (2.2)$$

Se si conoscesse Q^* , la politica ottima sarebbe immediata da ricavare: scegliere sempre $\operatorname{argmax}_a Q^*(s, a)$. Il problema del RL si riduce quindi, in buona parte, a stimare Q^* senza conoscerlo a priori.

2.2.3 Apprendimento model-based e model-free

Esiste una distinzione fondamentale tra due famiglie di algoritmi RL, a seconda che l'agente costruisca o meno una rappresentazione esplicita della dinamica dell'ambiente. Un algoritmo *model-based* apprende o riceve un modello approssimato di $P(s' | s, a)$ e $R(s, a)$, e lo usa per simulare le conseguenze di più sequenze di azioni prima di sceglierne una, senza eseguirle davvero nell'ambiente. Un algoritmo *model-free*, al contrario, non costruisce alcun modello della dinamica dell'ambiente, bensì stima direttamente Q^* dall'esperienza osservata, senza mai provare a prevedere esplicitamente lo stato successivo. Il vantaggio del model-free è la semplicità e la minore dipendenza dalla qualità di un modello che potrebbe essere difficile da apprendere accuratamente; lo svantaggio è un uso tipicamente meno efficiente dell'esperienza raccolta, poiché non la si riusa per simulare scenari mai osservati direttamente.

Questo lavoro adotta un approccio interamente model-free. Né il modello di riferimento (Wang et al., 2022) né le varianti implementate in questa tesi apprendono o usano un modello esplicito della dinamica del traffico. L'agente stima direttamente una funzione Q dall'esperienza simulata in CityFlow, con una famiglia di tecniche descritte nel resto di questa sezione.

2.2.4 Apprendimento online e offline

Una distinzione ortogonale a quella model-based/model-free appena vista riguarda da dove arriva l'esperienza usata per apprendere.

Un algoritmo *online* interagisce ripetutamente con l'ambiente durante l'addestramento, raccogliendo nuova esperienza con la politica corrente e usandola subito per migliorarsi, in un ciclo continuo di raccolta e apprendimento. Un algoritmo *offline*, al contrario, apprende da un insieme fisso di transizioni raccolto in precedenza, magari con una politica diversa o addirittura sconosciuta, senza alcuna ulteriore interazione con l'ambiente durante il training. Questo risulta essere un approccio necessario quando interagire con l'ambiente reale è costoso o rischioso. Un dettaglio che vale la pena chiarire subito è che un *replay buffer* come quello che il DQN (descritto a breve) introduce, non lo rende di per sé un metodo offline. Il buffer infatti si riempie continuamente di transizioni generate via via da una politica in evoluzione, con raccolta dati e apprendimento che procedono in parallelo, non in due fasi separate come negli algoritmi offline.

Questo lavoro è interamente online: ogni transizione usata per un aggiornamento proviene dalla simulazione in corso in CITYFLOW, mai da un insieme di dati raccolto in anticipo.

2.2.5 Ampiezza e profondità dell'aggiornamento

Una seconda distinzione, anch'essa ortogonale alle precedenti, riguarda il modo in cui si aggiorna una stima di valore combinando una ricompensa con la stima del valore in uno o più stati successivi. Due assi indipendenti classificano ogni aggiornamento, come mostrato in Figura 2.5.

Il primo asse riguarda l'**ampiezza** dell'aggiornamento. Un aggiornamento completo (*full backup* in figura) calcola l'aspettativa matematica considerando tutte le azioni possibili e tutti i possibili stati successivi s' , pesati dalla probabilità di transizione $P(s' | s, a)$ fornita da un modello noto dell'ambiente. Un aggiornamento campionato (*sample backup* in figura) si basa invece su una singola traiettoria o un'esperienza realmente osservata, senza calcolare tutte le diramazioni possibili.

Il secondo asse riguarda la **profondità** dell'aggiornamento. Un aggiornamento superficiale, o a corto raggio (*shallow backup* in figura), guarda solo a pochissimi passi nel futuro, nel caso di TD(0) un solo passo d'azione, e sostituisce il ritorno futuro con la stima già disponibile dell'ultimo passo considerato. Questa è una tecnica nota come *bootstrapping*, per cui si costruisce una stima a partire da un'altra stima invece che dall'esito osservato per intero. Un aggiornamento profondo, o a lungo raggio (*deep backup* in figura), tiene conto delle conseguenze fino alla fine dell'episodio, usando solo ricompense effettivamente realizzate e non stimate.

Le combinazioni dei due assi individuano quattro famiglie di metodi:

- **Completo e superficiale:** la *dynamic programming*, che calcola l'aspettativa esatta su tutti i successori s' ma si ferma dopo un solo passo, facendo bootstrapping sulla stima già disponibile del valore di s' invece di espandere ulteriormente l'albero.
- **Completo e profondo:** la *ricerca esaustiva*, che come la dynamic programming pesa ogni successore con la sua probabilità, ma che espande però l'intero albero delle conseguenze fino alla fine dell'episodio o a un orizzonte fissato. È il caso ideale in cui ci si potesse permettere di enumerare ogni possibile continuazione.
- **Campionato e profondo:** il *Monte Carlo* non usa $P(s' | s, a)$, osserva un'unica traiettoria effettivamente realizzata e ne percorre per intero le conseguenze fino alla fine dell'episodio, calcolando il ritorno effettivo $G_t = \sum_k \gamma^k r_{t+k}$ senza mai usare bootstrapping.
- **Campionato e superficiale:** il *Temporal Difference* (TD), che come Monte Carlo osserva una singola transizione campionata anziché un'aspettativa

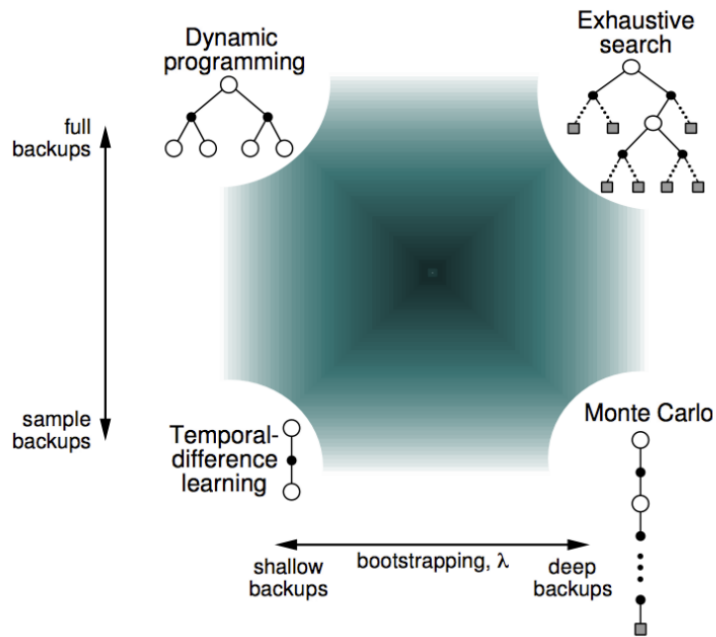


Figura 2.5: Lo spazio dei metodi di apprendimento e pianificazione, classificati secondo la profondità dell'aggiornamento (asse orizzontale, superficiale a sinistra, profondo a destra) e la sua ampiezza (asse verticale, completo in alto, campionato in basso); le etichette in figura usano i termini inglesi *shallow/deep backups* per l'asse orizzontale e *full/sample backups* per il verticale. I quattro angoli corrispondono a dynamic programming, ricerca esaustiva, Temporal Difference e Monte Carlo. Adattata da Sutton and Barto (2018).

sul modello, ma come la dynamic programming si ferma dopo un solo passo, facendo bootstrapping sulla stima corrente del valore dello stato successivo.

Nella sua versione più semplice di Temporal Difference, TD(0), l'aggiornamento della funzione valore-stato è

$$V(s_t) \leftarrow V(s_t) + \eta \left[(r_t + \gamma V(s_{t+1})) - V(s_t) \right],$$

dove il termine tra parentesi quadre è il *Temporal Difference error*, ovvero la discrepanza tra la stima corrente e un target calcolato guardando avanti di un solo passo. Il bootstrapping ha un costo, l'errore della stima usata come target si propaga nell'aggiornamento, ma un beneficio pratico decisivo rispetto a Monte Carlo: l'agente può aggiornarsi a ogni singolo passo, senza attendere la fine dell'episodio, un requisito essenziale per imparare da compiti molto lunghi o addirittura privi di una fine naturale. Il Q-learning descritto di seguito applica esattamente questo principio alla funzione valore-azione Q invece che alla sola funzione valore-stato V .

Questo lavoro si colloca nella casistica dell'aggiornamento campionato e superficiale, quello del Temporal Difference. L'assenza di un modello della dinamica del traffico (§2.2.3) esclude già di per sé la dynamic programming e la ricerca esaustiva, entrambe legate a un aggiornamento completo; inoltre la simulazione in CITY-FLOW procede per episodi lunghi e continui su intersezioni multiple, un contesto in cui attendere la fine dell'episodio per ogni aggiornamento, come farebbe Monte Carlo, sarebbe sia lento sia inutilmente rumoroso rispetto al bootstrap a un passo. Il Q-learning e il DQN che ne deriva, discussi di seguito, adottano infatti un aggiornamento TD, campionato e superficiale.

2.2.6 Q-learning e Deep Q-Network

Q-learning tabellare. Il **Q-learning** (Sutton and Barto, 2018) approssima Q^* applicando iterativamente l'Equazione 2.2 come regola di aggiornamento tramite bootstrapping: a ogni transizione osservata (s, a, r, s') , aggiusta il valore stimato $Q(s, a)$ verso il target $r + \gamma \max_{a'} Q(s', a')$, la stima corrente al passo successivo, invece di attendere il ritorno realizzato per intero a fine episodio come farebbe un metodo Monte Carlo. Il nome "tabellare" indica che questi aggiornamenti possono essere eseguiti considerando una tabella con una riga per ogni stato e una colonna per ogni azione. L'aggiornamento modifica una voce alla volta e ripetendo l'operazione su molte transizioni, ogni voce $Q(s, a)$ converge verso il vero ritorno atteso dell'eseguire a in s . Si ottiene in questo modo una politica ottima, senza il bisogno di indicazioni esplicite sulle azioni da intraprendere.

L'approccio tabellare ha però un limite rappresentato dalle dimensioni della tabella. Se lo spazio degli stati e lo spazio delle azioni sono continui o ad alta dimensionalità il numero di combinazioni possibili esplode, rendendo questo approccio impraticabile. Questo è il caso del controllo semaforico multi-agente, dove la dimensione dello spazio degli stati rende una tabella esplicita irrealizzabile sia da costruire sia da aggiornare.

Deep Q-Network (DQN). Mnih et al. (2015) risolvono il problema sostituendo la tabella con una rete neurale $Q(s, a; \theta)$ allenata ad approssimarne i valori: invece di una voce indipendente per ogni coppia (s, a) , la rete apprende una funzione continua che generalizza a stati mai visti interpolando tra stati simili osservati durante l'addestramento. I pesi θ vengono aggiornati per discesa del gradiente in modo che $Q(s, a; \theta)$ si avvicini allo stesso target di Bellman usato dal Q-learning tabellare, con la differenza che l'aggiornamento modifica l'intera rete invece di una singola voce, propagando l'informazione anche a stati simili mai osservati direttamente. È esattamente questa proprietà a rendere il DQN necessario in questo lavoro: lo spazio di stato del controllo semaforico multi-agente è troppo grande per una tabella, ma una rete neurale può generalizzare a partire da un numero gestibile di transizioni osservate durante la simulazione. Gli autori introducono due accorgimenti per stabilizzare questo schema, entrambi adottati anche in questo lavoro. Il primo è il *replay buffer*: una memoria in cui si accumulano le transizioni (s, a, r, s') osservate, da cui si campionano poi mini-batch in ordine casuale per l'addestramento, rompendo la correlazione temporale tra stati consecutivi di uno stesso episodio che violerebbe altrimenti l'assunzione di campioni indipendenti richiesta dalla discesa del gradiente stocastica. Come anticipato (§2.2.4), il buffer non cambia la natura online del metodo, in quanto si riempie di transizioni raccolte dalla politica corrente mentre l'addestramento procede, non da un insieme fisso raccolto in anticipo. Il secondo è la *target network* $Q(\cdot; \theta^-)$, una copia dei pesi della rete online sincronizzata solo periodicamente: calcolare il target $r + \gamma \max_{a'} Q(s', a'; \theta^-)$ con questa copia invece che con la rete online stessa, che cambia a ogni passo di gradiente, evita che il bersaglio dell'ottimizzazione si sposti insieme ai pesi che lo determinano, un problema noto come *moving target*.

2.2.7 Esplorazione ε -greedy

Un agente completamente *greedy*, che segue cioè sempre e solo l'azione a valore Q stimato più alto, non raccoglierebbe mai esperienza sulle conseguenze di azioni alternative, rischiando di convergere su una politica sub-ottima per via di una mancata

esplorazione. Ad esempio, se nelle prime fasi dell’addestramento un’azione appare leggermente migliore di un’altra, un agente completamente *greedy* la prediligerebbe sistematicamente, impedendo di fatto all’agente di scoprire il potenziale di strategie alternative. La tecnica più comune per bilanciare sfruttamento (*exploitation*, scegliere l’azione ritenuta migliore) ed esplorazione (*exploration*, raccogliere informazione su azioni meno note) è ϵ -greedy (Sutton and Barto, 2018): a ogni passo di decisione, con probabilità ϵ l’agente sceglie un’azione uniformemente a caso, e con probabilità $1 - \epsilon$ sceglie $\arg\max_a Q(s, a; \theta)$. Il valore di ϵ tipicamente decresce nel corso dell’addestramento, da un valore iniziale vicino a 1 (quasi solo esplorazione, quando la stima di Q è ancora inaffidabile) verso un valore finale piccolo ma non nullo (quasi solo sfruttamento, mantenendo comunque un minimo di esplorazione residua).

2.2.8 Estensioni usate in questo lavoro

Il DQN di base ammette diverse estensioni indipendenti, ciascuna delle quali corregge una specifica fonte di instabilità o inefficienza, discusse di seguito:

- Il **Double DQN** (van Hasselt et al., 2016) disaccoppia la selezione dell’azione dalla sua valutazione, usando la rete online per scegliere l’azione migliore e la target network, già introdotta sopra, solo per valutarla: $y = r + \gamma Q(s', \arg\max_{a'} Q(s', a'; \theta^-); \theta^-)$. Poiché la stessa rete che sceglie l’azione con il valore stimato più alto tende anche a sovrastimarne il valore, usare la rete online per scegliere e la target network per valutare riduce questo bias sistematico di sovrastima tipico del DQN standard, che userebbe la stessa rete per entrambi i passi.
- Il **Prioritized Experience Replay** (PER, Schaul et al., 2016) sostituisce il campionamento uniforme dal replay buffer con un campionamento proporzionale al *Temporal Difference error* δ_i . Le transizioni che la rete predice peggio vengono ricampionate più spesso, concentrando la capacità di apprendimento dove serve di più. Assegnando a ciascuna transizione una priorità $p_i = |\delta_i| + \zeta$, con ζ una piccola costante che evita una probabilità nulla, la probabilità di campionamento è $P(i) = p_i^\alpha / \sum_k p_k^\alpha$, con $\alpha \in [0, 1]$ iperparametro che ne controlla la dipendenza dall’errore. Per correggere il bias introdotto da un campionamento non uniforme, la loss viene pesata tramite coefficienti di *importance sampling* $w_i = (N \cdot P(i))^{-\beta}$, dove N è la dimensione del buffer e $\beta \in [0, 1]$ un iperparametro che cresce nel corso dell’addestramento verso 1, aumentando progressivamente il grado di correzione applicato.

- La **Huber loss** (o *smooth L1*) sostituisce la funzione di loss data dall'errore quadratico medio (MSE) con una funzione quadratica vicino allo zero e lineare lontano da esso, risultando meno sensibile agli outlier prodotti da transizioni con errore di stima molto grande (con MSE, un singolo errore enorme domina il gradiente dell'intero batch; con Huber il suo contributo cresce solo linearmente):

$$L_{\kappa}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{se } |y - \hat{y}| \leq \kappa \\ \kappa \left(|y - \hat{y}| - \frac{1}{2}\kappa \right) & \text{altrimenti} \end{cases} \quad (2.3)$$

Tipicamente $\kappa = 1$.

2.2.9 Apprendimento multi-agente

Quanto descritto finora assume un solo agente che interagisce con un solo ambiente. Quando più agenti operano nello stesso ambiente, ciascuno osservando solo una porzione dello stato globale e influenzando con le proprie azioni anche ciò che gli altri agenti osservano e ricevono come ricompensa, il problema si generalizza da MDP a *processo decisionale di Markov decentralizzato e parzialmente osservabile* (Dec-POMDP). Il termine "decentralizzato" si riferisce al fatto che non esiste un'unica azione scelta da un solo decisore, ma N azioni scelte indipendentemente da N agenti; "parzialmente osservabile" indica invece che ogni agente vede solo il proprio stato locale, non lo stato globale completo dell'ambiente. Nel contesto di questo lavoro, la decentralizzazione si concretizza nel fatto che ogni intersezione decide autonomamente la propria fase, mentre l'osservabilità parziale si concretizza nel campo visivo limitato di ciascuna intersezione (Capitolo 3), che esclude deliberatamente dallo stato osservato ciò che accade agli altri incroci.

Il multi-agente introduce una difficoltà che il caso a singolo agente non ha: la *non stazionarietà*. L'equazione di Bellman e l'intera teoria del Q-learning assumono che l'ambiente risponda in modo coerente nel tempo alla stessa azione nello stesso stato; ma se più agenti apprendono simultaneamente, ciascuno sta anche cambiando il proprio comportamento mentre gli altri lo osservano, cosicché per ogni singolo agente l'ambiente (che include il comportamento degli altri agenti) appare cambiare regole nel tempo, anche se le vere dinamiche fisiche sottostanti restano identiche.

Una seconda difficoltà è l'*assegnazione del merito* (*credit assignment*), per la quale quando più agenti contribuiscono a un solo risultato osservato, non è ovvio quale frazione di quel risultato attribuire a ciascuno. Tale problema però non si pone se ogni agente riceve una ricompensa calcolata esclusivamente sul proprio stato locale,

così come avviene in questo lavoro. Una versione attenuata resta comunque presente: lo stato locale di un agente, e quindi la sua stessa ricompensa, dipende in parte dalle scelte degli altri agenti, per cui l'effetto della propria azione e quello dell'azione altrui rimangono in parte intrecciati anche con una ricompensa puramente locale.

Due schemi di coordinamento ricorrono in letteratura (Li, 2018): l'*addestramento centralizzato con esecuzione decentralizzata* (CTDE), in cui gli agenti condividono informazione globale solo durante il training per poi decidere localmente, e l'*addestramento ed esecuzione entrambi decentralizzati* (DTDE), in cui l'informazione resta sempre e solo locale, anche durante l'addestramento. Una scelta indipendente da questa è se i diversi agenti condividano o meno gli stessi pesi (*parameter sharing*): dividerli riduce il numero di parametri totali e permette a un agente di beneficiare dell'esperienza raccolta da tutti gli altri, a costo di una minore capacità di specializzarsi su condizioni locali peculiari, a meno che il meccanismo non trovi un altro modo per differenziare il comportamento tra agenti pur condividendo i pesi.

Questo lavoro rappresenta un sistema multi-agente in cui ogni intersezione della rete stradale è un agente a sé, con un proprio stato s_i^t , una propria azione a_i^t (la fase semaforica scelta) e una propria ricompensa r_i^t , calcolati indipendentemente per ogni intersezione i (Capitolo 3). L'architettura è di tipo DTDE, con scambio di informazione locale tra intersezioni vicine (§2.4), e adotta il *parameter sharing*: tutte le intersezioni condividono la stessa rete Q , ma un meccanismo di meta-learning genera pesi diversi per nodi con caratteristiche locali diverse, così da attenuare proprio il costo della condivisione appena descritto. Va precisato che questa condivisione, insieme al fatto che l'aggiornamento del gradiente sia calcolato in un solo passo che aggrega la loss di tutte le intersezioni (Capitolo 3), non rende l'addestramento centralizzato nel senso di CTDE: ciò che definisce quello schema è l'uso, durante il solo training, di informazione non disponibile in esecuzione, mentre qui ogni termine della loss usa esclusivamente lo stato e la ricompensa locali della propria intersezione, esattamente gli stessi disponibili in esecuzione.

Quando la rete Q include un modulo ricorrente, condividere i pesi tra le intersezioni pone un'ulteriore difficoltà legata allo stato ricorrente campionato dal replay buffer, discussa in §2.3 dopo aver introdotto le reti ricorrenti.

2.3 Reti Neurali Ricorrenti

Una rete neurale *feedforward* calcola il proprio output come funzione del solo input corrente. Dunque ogni esempio è trattato in modo indipendente dai precedenti, e la rete non conserva alcuna memoria di cosa abbia visto in passato. Questa assunzione

fallisce ogni volta che i dati hanno una struttura sequenziale in cui l'ordine e la storia contano quanto il singolo valore osservato (testo, audio, serie temporali, o lo stato di un'intersezione stradale). Le *reti neurali ricorrenti* (RNN) affrontano il problema introducendo un ciclo: a ogni istante t , un nodo riceve in ingresso non solo il dato corrente x^t ma anche il proprio stato nascosto al passo precedente h^{t-1} ,

$$h^t = \sigma(W_{hx}x^t + W_{hh}h^{t-1} + b_h), \quad (2.4)$$

così che l'informazione rilevante di tutta la sequenza osservata fino a t possa, in linea di principio, persistere in h^t . Lo stesso insieme di pesi W_{hx}, W_{hh}, b_h viene riapplicato a ogni istante di tempo, invece di avere pesi diversi per ogni passo. "Srotolando" la rete lungo l'asse del tempo, cioè disegnando esplicitamente una copia dei nodi per ogni istante $t = 1, 2, \dots$ collegata alla successiva (Figura 2.6), ogni passo diventa uno strato di una rete feedforward molto profonda a pesi condivisi, addestrabile con la stessa backpropagation vista in precedenza applicata lungo la sequenza anziché lungo i layer: questo algoritmo prende il nome di *backpropagation through time* (BPTT).

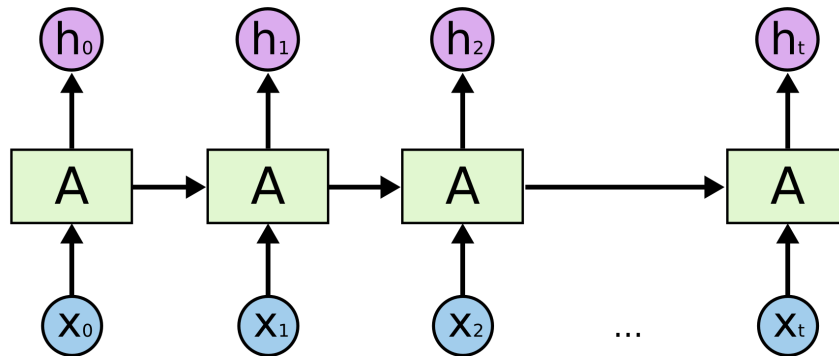


Figura 2.6: La rete ricorrente srotolata lungo il tempo: il modulo A rappresenta la funzione ricorrente, ad esempio l'Equazione 2.4, riapplicata con gli stessi pesi a ogni istante. Fonte: adattata da una discussione su Stack Exchange.³

Il problema del gradiente. Una rete ricorrente addestrata su sequenze lunghe soffre di un problema strutturale: il gradiente propagato all'indietro attraverso molti passi temporali tende ad annullarsi (*vanishing gradient*) o a divergere (*exploding gradient*) a seconda che i pesi ricorrenti abbiano modulo minore o maggiore di uno, poiché il gradiente si ottiene moltiplicando ripetutamente per lo stesso fattore a ogni passo srotolato, esattamente come una potenza c^n tende a zero o esplode al crescere di n a seconda che $|c|$ sia minore o maggiore di uno. Nella pratica, questo rende di

fatto impossibile apprendere dipendenze che coprono più di poche decine di passi con una RNN semplice.

Long Short-Term Memory (LSTM). L'LSTM (Hochreiter and Schmidhuber, 1997; Lipton et al., 2015) risolve il problema sostituendo il semplice nodo ricorrente con una *cella di memoria*: uno stato interno c^t collegato a se stesso da un percorso a peso fisso unitario (il *constant error carousel*), attraverso cui il gradiente può fluire per molti passi senza annullarsi né esplodere. L'accesso a questo stato è regolato da tre *gate*, unità sigmoidali il cui valore in $[0, 1]$ modula quanta informazione passi attraverso di esse (Figura 2.7).

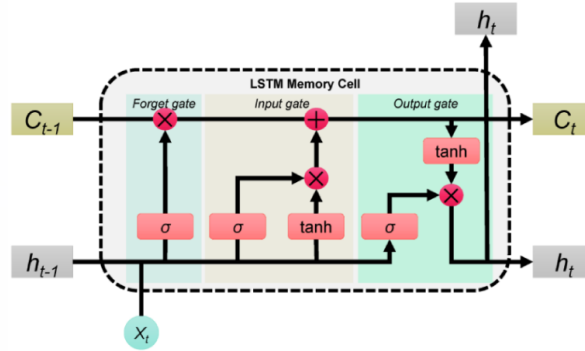


Figura 2.7: La cella di memoria dell'LSTM: il forget gate decide quanto dello stato precedente mantenere, l'input gate quanto del nuovo contenuto candidato incorporare, l'output gate quanto dello stato risultante esporre come output. Fonte: adattata da dida.do.⁴

A livello formale, con pesi $W_g \in \mathbb{R}^{2d_1 \times d_1}$ e bias $b_g \in \mathbb{R}^{d_1}$ condivisi da ogni nodo, i tre gate e il contenuto candidato $g \in \{\text{forget}, \text{input}, \text{output}, \text{cell}\}$ si calcolano come:

$$\begin{bmatrix} f^t \\ \iota^t \\ o^t \\ g^t \end{bmatrix} = \begin{bmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{bmatrix} \left(W_{\{f, \iota, o, g\}}^\top [x^t \parallel h^{t-1}] + b_{\{f, \iota, o, g\}} \right), \quad (2.5)$$

$$c^t = f^t \odot c^{t-1} + \iota^t \odot g^t,$$

$$h^t = o^t \odot \tanh(c^t);$$

il *forget gate* f decide quanto dello stato precedente dimenticare, l'*input gate* ι quanto del nuovo contenuto candidato g incorporare nello stato aggiornato, e l'*output gate* o quanto dello stato risultante esporre come output della cella. Il contenuto

candidato g non è esso stesso un gate, cioè un moltiplicatore in $[0, 1]$, ma il nuovo valore, calcolato con $\tanh$ e quindi in $[-1, 1]$, che i gate regolano. Un valore di forget gate vicino a 1 lascia passare l'informazione pregressa quasi inalterata anche dopo molti passi; uno vicino a 0 la cancella, permettendo alla rete di "dimenticare" deliberatamente informazione non più rilevante.

Gated Recurrent Unit (GRU). Una variante più leggera dell'LSTM, la **GRU** (Cho et al., 2014), fonde il *forget gate* e l'*input gate* in un unico *update gate* e rinuncia allo stato di cella c^t separato, esponendo un solo stato nascosto h^t sia come memoria interna sia come output. Si hanno così meno parametri da addestrare e un grafo computazionale più semplice, a fronte di una capacità di rappresentazione leggermente inferiore rispetto ai quattro gate distinti dell'LSTM. Le due varianti raggiungono nella pratica prestazioni simili su molti compiti, e la scelta tra loro è spesso guidata più dalla convenzione di un'architettura di riferimento che da un vantaggio netto dell'una sull'altra: questo lavoro adotta l'LSTM per coerenza con il modello di riferimento (Wang et al., 2022).

Replay ricorrente e disallineamento dello stato. Quando la rete Q include un modulo ricorrente come l'LSTM appena descritto, campionare dal replay buffer (§2.2) transizioni isolate anziché intere sequenze pone un problema specifico: lo stato ricorrente (h, c) usato durante l'addestramento non coincide necessariamente con quello osservato durante la raccolta dati. Per una transizione campionata isolatamente, l'unica scelta praticabile è inizializzare (h, c) a zero, un disallineamento rispetto allo stato che la rete aveva effettivamente accumulato in quel punto della sequenza originale, un problema noto come *hidden state staleness*. Seguendo Kaptrowski et al. (2019) (R2D2), il campionamento si esegue allora su intere sequenze di lunghezza L , dedicando i primi passi della sequenza (fase di *burn-in*) esclusivamente alla ricostruzione di uno stato ricorrente plausibile, senza calcolare gradiente, per poi eseguire la BPTT sui passi restanti.

2.4 Grafi e Graph Neural Network

2.4.1 Cos'è un grafo

Un *grafo* $G = (V, E)$ è una struttura matematica composta da un insieme di *nodi* V e un insieme di *archi* E , dove ogni arco collega una coppia di nodi. È lo strumento standard per rappresentare entità e le relazioni tra loro, quando quelle relazioni non hanno una struttura regolare o sequenziale. Ne sono esempi una rete di amicizie

tra persone, le pagine web collegate da link, le molecole i cui atomi sono legati da legami chimici, e, in questo lavoro, una rete stradale, dove i nodi sono le intersezioni e gli archi le strade che le collegano direttamente.

Un arco può essere *diretto* (se indica un verso) o *non diretto* (se indica una relazione simmetrica, Figura 2.8); può inoltre avere un *peso* associato, un numero che quantifica l'intensità o il costo della relazione.

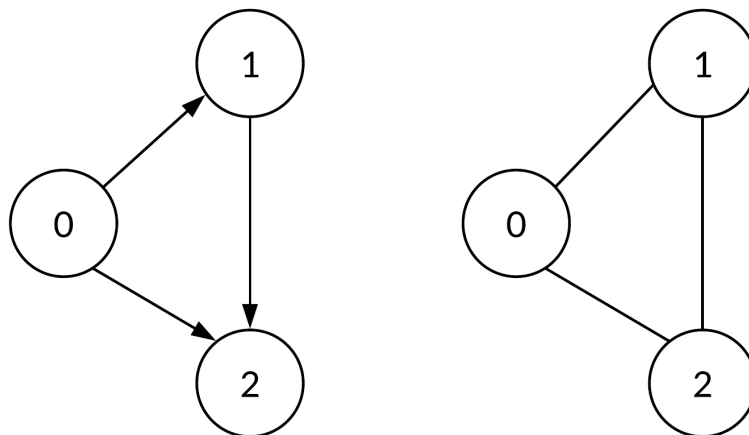


Figura 2.8: Esempi di grafo diretto (sinistra) e non diretto (destra).

L'insieme dei nodi collegati a un nodo i tramite un arco si chiama *vicinato* di i , indicato $\mathcal{N}(i)$; il numero di archi incidenti su i è il suo *grado*. Una sequenza di nodi collegati uno al successivo da un arco è un *cammino*; la lunghezza del cammino più breve tra due nodi qualunque è la loro *distanza* sul grafo, misurata in numero di archi. Un grafo con N nodi si può rappresentare in forma matriciale con la *matrice di adiacenza* $A \in \{0, 1\}^{N \times N}$, dove $A_{iu} = 1$ se e solo se esiste un arco tra i e u (o il peso dell'arco, se il grafo è pesato, invece del semplice 1). È spesso utile aggiungere a ogni nodo un *self-loop*, un arco da un nodo verso se stesso ($A_{ii} = 1$), cosicché un nodo sia incluso nel proprio stesso vicinato: la notazione $\hat{A} = A + I$ indica questa aggiunta e ricorre nelle equazioni seguenti.

Un piccolo esempio: la matrice di adiacenza senza pesi del grafo non diretto in Figura 2.8 è data da

$$A = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}.$$

Con l'aggiunta dei self-loop, $\hat{A} = A + I$ ha inoltre un 1 su ogni elemento della diagonale.

Nella rete stradale di questo lavoro, ogni nodo i del grafo porta con sé un vettore di caratteristiche (o *embedding*) che ne descrive lo stato corrente, ad esempio quanti veicoli attendono su ciascuna corsia. Il problema che le *graph neural network* risolvono è come far sì che l’embedding di un nodo tenga conto anche di ciò che accade nei nodi a lui collegati, non solo del proprio stato isolato.

2.4.2 Message passing

Le *Graph Neural Network* (GNN) affrontano questo problema con un principio comune, il *message passing* (Wu et al., 2021; Bacciu et al., 2020). A ogni iterazione (o *layer*) l , ogni nodo i con rappresentazione corrente $h_i^{(l)}$ esegue due operazioni in successione: prima un passo di *aggregazione*, che combina i messaggi ricevuti da ciascun vicino $u \in \mathcal{N}(i)$ in un unico vettore di messaggio m_i ,

$$m_i = \text{AGGREGATE}(\{h_u^{(l)} : u \in \mathcal{N}(i)\}),$$

poi un passo di *aggiornamento*, che combina il messaggio aggregato con la rappresentazione precedente dello stesso nodo per produrre quella nuova,

$$h_i^{(l+1)} = \text{UPDATE}(h_i^{(l)}, m_i).$$

Tutte le GNN discusse in questa tesi, incluse le formulazioni di seguito, sono istanze di questo stesso schema generale, mostrato in Figura 2.9, che differiscono solo nella scelta concreta di AGGREGATE e UPDATE. Nella GCN, AGGREGATE è una media pesata dalla sola topologia; nella GAT, un peso appreso in base al contenuto dei due nodi coinvolti. Impilando più layer, l’informazione può viaggiare oltre il solo vicinato immediato: dopo k layer, la rappresentazione di un nodo dipende (indirettamente) da tutti i nodi raggiungibili entro k archi di distanza, poiché il messaggio ricevuto da un vicino al layer l incorpora già, ricorsivamente, i messaggi che quel vicino aveva ricevuto dai propri stessi vicini al layer $l - 1$.

Graph Convolutional Network (GCN). Kipf and Welling (2017) propongono la formulazione più basilare di message passing tra quelle qui discusse. La rappresentazione di ciascun nodo si aggiorna mediando quelle dei vicini con un peso fisso, determinato unicamente dalla topologia locale (quanti vicini ha un nodo), non dal contenuto delle rappresentazioni stesse. Indicando con $\hat{A} = A + I$ la matrice di adiacenza con self-loop e con $\hat{D}$ la matrice diagonale dei gradi corrispondente ($\hat{D}_{ii} = \sum_u \hat{A}_{iu}$), la regola di propagazione di un singolo layer si scrive, in forma

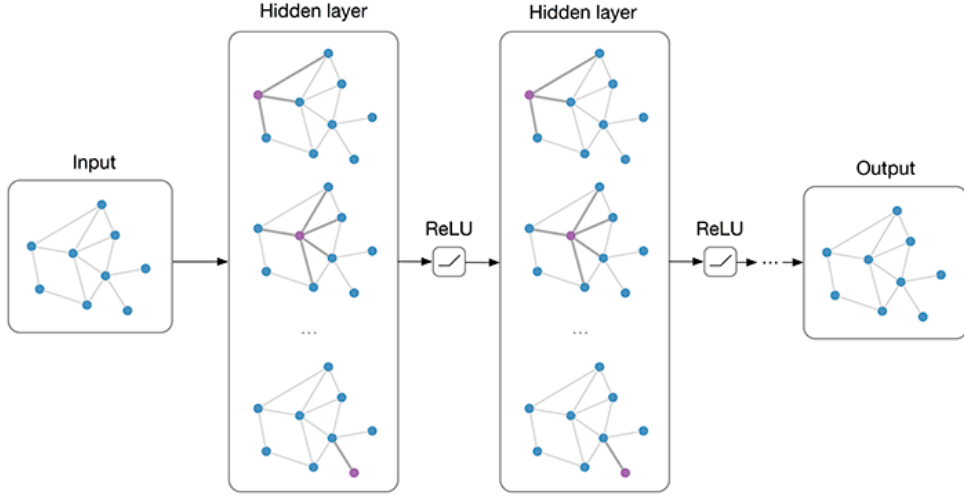


Figura 2.9: Lo schema generale di message passing impilato su più layer: a ogni layer ciascun nodo aggrega le rappresentazioni dei propri vicini e aggiorna la propria, prima di applicare la non linearità σ e passare al layer successivo. Fonte: adattata da Towards AI.⁵

matriciale:

$$H^{(l+1)} = \sigma\left(\hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2} H^{(l)} W^{(l)}\right) \quad (2.6)$$

dove $H^{(l)} \in \mathbb{R}^{N \times d_l}$ raccoglie le rappresentazioni di tutti gli N nodi del grafo al layer l , $W^{(l)}$ è la matrice di pesi condivisa da ogni nodo e σ una non linearità puntuale. Per un singolo nodo i , la normalizzazione simmetrica $\hat{D}^{-1/2} \hat{A} \hat{D}^{-1/2}$ equivale a pesare il contributo di ogni vicino u con $1/\sqrt{\hat{d}_i \hat{d}_u}$:

$$h_i^{(l+1)} = \sigma\left(\sum_{u \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{\hat{d}_i \hat{d}_u}} W^{(l)} h_u^{(l)}\right) \quad (2.7)$$

un peso identico per ogni arco del vicinato di i , funzione solo del grado dei due nodi coinvolti e mai del loro contenuto. È, per costruzione, il termine di paragone più naturale rispetto a cui misurare cosa aggiunge un meccanismo di attenzione appreso come quello discusso di seguito.

Graph Attention Network (GAT). Veličković et al. (2018) sostituiscono il peso fisso della GCN con un coefficiente di attenzione appreso, specifico per ciascuna coppia di nodi collegati: il contributo di un vicino non dipende più soltanto dalla topologia, ma anche dal suo contenuto. Dato un nodo i con vicinato $\mathcal{N}(i)$, embedding di input h_i , matrice di proiezione condivisa W e vettore di attenzione $\vec{a}$:

$$e_{iu} = \text{LeakyReLU}\left(\vec{a}^\top [W h_i \parallel W h_u]\right), \quad \alpha_{iu} = \frac{\exp(e_{iu})}{\sum_{u' \in \mathcal{N}(i)} \exp(e_{iu'})} \quad (2.8)$$

$$h'_i = \sigma \left(\sum_{u \in \mathcal{N}(i)} \alpha_{iu} W h_u \right) \quad (2.9)$$

dove $\parallel$ denota la concatenazione: e_{iu} è un punteggio di compatibilità grezzo tra i e u , che il *softmax* sul vicinato trasforma in un coefficiente $\alpha_{iu} \in [0, 1]$ che somma a uno su tutti i vicini di i , esattamente come una distribuzione di probabilità, come illustrato in Figura 2.10.

Graph Attention Networks

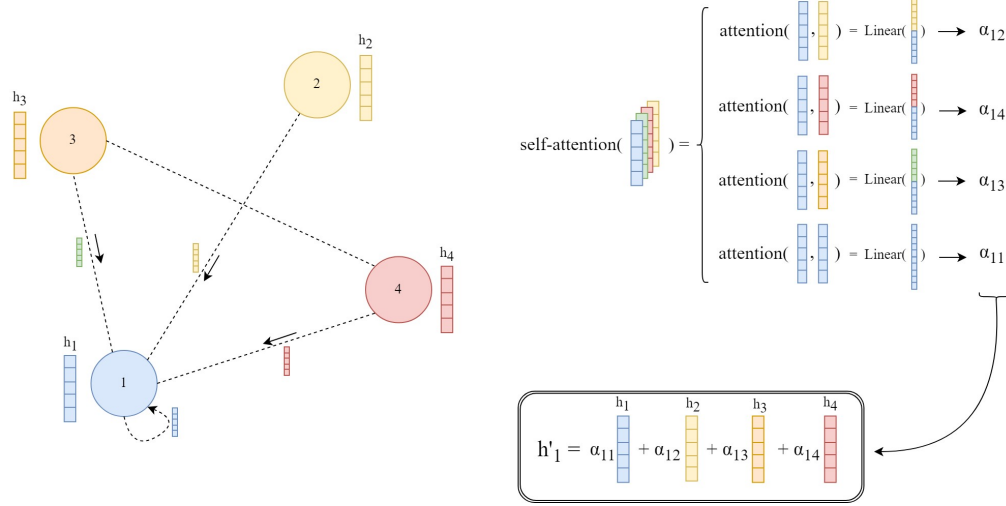


Figura 2.10: Il calcolo dell'attenzione per il nodo 1 rispetto al proprio vicinato $\{1, 2, 3, 4\}$ (con self-loop incluso): ogni coppia (h_1, h_u) produce un coefficiente α_{1u} , usato poi per combinare linearmente gli embedding dei vicini nella nuova rappresentazione h'_1 . La figura, semplificata a scopo illustrativo, accorpa in un unico passo "Linear" il calcolo di compatibilità e la normalizzazione softmax descritti nel testo, e omette la proiezione condivisa W e la non linearità σ . Fonte: adattata da Data Science at Home.⁶

Con K teste di attenzione indipendenti (*multi-head attention*), ciascuna con i propri pesi W e $\vec{a}$, gli output si concatenano nei layer intermedi e si mediano nell'ultimo, secondo lo schema introdotto per l'attenzione da Vaswani et al. (2017). In questo modo teste diverse possono imparare a specializzarsi su aspetti diversi della relazione tra due nodi. A differenza della GCN, il coefficiente α_{iu} consente a un nodo di ponderare diversamente i propri vicini in base al loro contenuto, non solo alla loro posizione nel grafo: una proprietà rilevante per un nodo i i cui vicini abbiano importanza marcatamente diversa, come nel caso di un incrocio stradale in cui passano un'arteria principale e una strada secondaria. La funzione di compatibilità additiva appena descritta, per quanto la più nota, non è l'unica possibile: il Capitolo 3 (Meta-GAT) ne impiega una variante strutturalmente diversa, con Query, Key e Value esplicite e distinte al posto della proiezione W condivisa, e un prodotto

scalare al posto della concatenazione.

2.4.3 Invarianza ed equivarianza per permutazione

Sia l'aggregazione della GCN sia quella della GAT condividono due proprietà distinte, che vale la pena separare.

La prima è ciò che permette a queste reti di funzionare su grafi di dimensione diversa senza dover riaddestrare nulla: i pesi W sono condivisi da ogni nodo, e l'operatore AGGREGATE (una somma pesata, sia essa uniforme come nella GCN o per attenzione come nella GAT) accetta un numero qualsiasi di vicini. Il numero di parametri del layer non dipende quindi dal numero di nodi N del grafo, una condizione necessaria perché lo stesso modello, addestrato su un grafo di una certa dimensione, possa essere applicato a un grafo di dimensione diversa.

La seconda è l'*invarianza per permutazione* dell'operatore di aggregazione: una somma non dipende dall'ordine in cui i suoi addendi vengono elencati, quindi il messaggio aggregato m_i di un nodo i non dipende dall'ordine in cui i vicini di i vengono numerati o elencati, ma solo da quali essi siano. È ciò che distingue strutturalmente una GNN da una rete feedforward applicata a un vettore di caratteristiche di lunghezza fissa. Una MLP a cui si desse in input, in un qualche ordine arbitrario, le caratteristiche di ciascun nodo del grafo concatenate assegnerebbe implicitamente un significato specifico a ciascuna posizione del vettore (il primo blocco di ingressi, il secondo, ...) e produrrebbe quindi un output diverso se semplicemente si rinumerassero i nodi, pur descrivendo esattamente la stessa rete stradale. Wei et al. (2019) osservano esattamente questo limite per gli schemi di coordinamento basati su una lista ordinata di vicini.

Applicando l'invarianza per permutazione a ogni nodo del grafo simultaneamente, l'intero layer di message passing risulta *equivariante per permutazione*: rinumerare i nodi in ingresso rinumererà allo stesso modo le rappresentazioni prodotte in uscita, invece di produrre un risultato arbitrariamente diverso (Bronstein et al., 2021). È questa equivarianza, insieme all'indipendenza del numero di parametri da N , a rendere possibile la generalizzazione a topologie di rete mai viste che il Capitolo 4 conduce su MetaSTGAT e sulle sue varianti.

2.4.4 Reti spatio-temporali su grafo

Molti grafi non sono statici: la topologia resta fissa, ma le caratteristiche osservate a ciascun nodo evolvono nel tempo, e le due dimensioni, spaziale e temporale, risultano intrecciate, poiché il valore osservato a un nodo in un dato istante è correlato sia

con i valori passati allo stesso nodo sia con quelli ai nodi limitrofi. La famiglia di modelli che combina un layer di propagazione sul grafo (GCN o GAT, appena visti) con una componente ricorrente (LSTM, §2.3) sull'asse temporale prende il nome di *spatiotemporal graph neural network* (STGNN), un termine ormai consolidato nella letteratura sul forecasting di serie temporali multivariate correlate (Cini et al., 2025). MetaSTGAT, descritto nel dettaglio nel Capitolo 3, costituisce un esempio di questa famiglia applicata al controllo semaforico anziché alla previsione: un layer di *graph attention* cattura la dipendenza spaziale tra intersezioni limitrofe, un layer ricorrente (LSTM) ne cattura l'evoluzione temporale, e i due vengono addestrati congiuntamente end-to-end.

2.4.5 Pesì generati da meta-learning

Le strutture neurali introdotte, GCN e GAT, nella loro formulazione originale, condividono gli stessi pesi su tutti i nodi del grafo: un'assunzione ragionevole quando i nodi sono sufficientemente omogenei, meno quando, come in una rete stradale reale, la topologia locale di ciascun nodo (numero di strade entranti, presenza di un'arteria, posizione periferica o centrale) può differire sensibilmente da nodo a nodo. Un *hypernetwork* (Ha et al., 2016) affronta il problema generando i pesi di un layer non come parametri fissi appresi direttamente, bensì come output di una seconda rete, condizionata sulle caratteristiche locali del nodo: è di fatto una via di mezzo tra pesi completamente condivisi e pesi completamente indipendenti per ogni nodo, che riduce il costo del parameter sharing senza rinunciare del tutto ai suoi benefici. Nel contesto di questo lavoro, un *hypernetwork* condizionato su caratteristiche locali dell'intersezione è esattamente il meccanismo che la letteratura sul controllo semaforico chiama *meta-learning* quando applicato a questo compito: i due termini descrivono lo stesso principio, il secondo è quello più diffuso nei lavori discussi in §2.5, dove si colloca esplicitamente MetaSTGAT.

2.4.6 Propagazione a lungo raggio

Sia la GCN sia la GAT propagano informazione a un solo passo di distanza (un arco nel grafo) per layer: raggiungere un vicino a distanza k richiede impilarne altrettanti, con un costo di parametri crescente e un rischio noto di *over-smoothing* (Li et al., 2018), per il quale le rappresentazioni dei nodi divengono troppo omogenee al crescere della profondità (ogni nodo finisce per "vedere" praticamente tutto il grafo, perdendo la capacità di distinguersi dai propri vicini). Meccanismi più recenti, tra cui SONAR (Trenta et al., 2025), affrontano il problema della propagazione a lungo

raggio secondo un principio diverso, ispirato ai sistemi dinamici oscillatori piuttosto che alla concatenazione sequenziale di layer. Il sistema si comporta come una rete di masse collegate da molle e smorzatori, una massa per nodo del grafo. Raccolto lo stato di tutti gli N nodi al tempo continuo t in $X(t) \in \mathbb{R}^{N \times d}$, la sua evoluzione è governata da un'equazione del second'ordine, un'accelerazione e non una semplice velocità:

$$\ddot{X}(t) = -\mathcal{L}^a X(t) W - D(X(t)) \odot \dot{X}(t) + F(X(t)) \quad (2.10)$$

Tre termini, tre ruoli distinti. $\mathcal{L}^a$ è un Laplaciano di grafo pesato da una conduttanza $a_{ui} \geq 0$ per arco, che tira lo stato di un nodo verso quello dei suoi vicini $u \in \mathcal{N}(i)$, come una molla. $D(\cdot) \geq 0$ è un termine dissipativo che agisce sulla velocità $\dot{X}(t)$, frenando il movimento come un attrito. $F(\cdot)$ è una forzante esterna, che inietta energia dall'esterno del sistema. Riscritta come sistema del prim'ordine, con velocità ausiliaria $V = \dot{X}$, e discretizzata con passo h su L iterazioni, l'aggiornamento a ogni iterazione diventa:

$$\begin{aligned} (\mathcal{L}^a X)_i &= \sum_{u \in \mathcal{N}(i)} a_{ui} (X_i - X_u), \\ V &\leftarrow V - h(\mathcal{L}^a X + D(X) \odot V - F(X)), \\ X &\leftarrow X + h \tanh(V) \end{aligned} \quad (2.11)$$

Le L iterazioni condividono sempre gli stessi pesi a_{ui} , $D(\cdot)$ e $F(\cdot)$, per cui, a differenza di uno stack di L layer indipendenti come GCN o GAT, il costo di parametri di questo meccanismo resta costante al crescere di L , un vantaggio strutturale dichiarato dagli stessi autori.

2.5 Traffic Signal Control come problema

2.5.1 Terminologia e metriche

Il *traffic signal control* (TSC), o controllo semaforico, decide periodicamente quali flussi veicolari autorizzare a un incrocio. Il vocabolario di base, comune a questo capitolo e ai Capitoli 3 e 4, è il seguente.

Ogni via che affluisce a un incrocio è divisa in *corsie*, ciascuna tipicamente dedicata a una o più svolte (diritto, sinistra, destra). Un *movimento* è una coppia ordinata di corsie, una in ingresso e una in uscita, che un veicolo può percorrere attraversando l'incrocio; una *fase* semaforica autorizza simultaneamente un sottoinsieme di movimenti compatibili tra loro, cioè le cui traiettorie non si intersecano. Il

controllore osserva lo stato dell'incrocio e sceglie una fase a ogni *step di decisione*, un intervallo di tempo fissato che si ripete per l'intera durata della simulazione.

Valutare un controllore richiede di misurare cosa succede ai veicoli. Il *tempo di percorrenza* (*travel time*) di un veicolo è il tempo trascorso tra il suo ingresso e la sua uscita dalla rete; l'*attesa* di un veicolo, in un dato istante, è il tempo per cui si trova fermo o comunque non ancora oltre l'incrocio a cui si sta avvicinando. Il *throughput* di un incrocio o di una rete è invece il numero di veicoli che riescono ad attraversarli nell'unità di tempo, una nozione teorica distinta dalla percentuale di veicoli che completano effettivamente il proprio percorso in una simulazione di durata finita, misurata nel Capitolo 4. Le definizioni precise di queste grandezze, e le metriche che questo lavoro ne ricava, sono rimandate ai Capitoli 3 e 4, dove vengono usate per la prima volta in forma quantitativa.

2.5.2 Due generazioni di controllo

Il TSC si articola in due approcci, in ordine crescente di complessità e, storicamente, di introduzione (Shams et al., 2018).

Controllo a tempi fissi. Un piano semaforico ciclico ripete all'infinito la stessa sequenza di fasi, ciascuna di durata prestabilita in base a stime di traffico medie e mai più rivista in tempo reale; nella sua forma più diffusa nei sistemi reali, la ripartizione del verde tra le fasi e la durata complessiva del ciclo possono variare per fascia oraria, ma restano fissate a tavolino e non rispondono al traffico osservato istante per istante. FIXED-TIME, la baseline di questo lavoro (Capitolo 3), ne è la forma più semplice: cicla le fasi in ordine fisso, cambiando fase a ogni step di decisione, senza alcun parametro da configurare oltre all'ordine del ciclo. È estremamente semplice da implementare ed è tuttora la spina dorsale di gran parte dei sistemi semaforici reali: negli Stati Uniti, ad esempio, una larga proporzione delle intersezioni semaforizzate resta priva di sensori di rilevamento ed è tuttora controllata a tempi fissi, proprio per gli alti costi di installazione e manutenzione dei rilevatori (Wang et al., 2024). Non richiede infatti sensori stradali in tempo reale né potenza computazionale per decidere. I suoi svantaggi principali sono la rigidità e l'incapacità di adattarsi alle fluttuazioni improvvise del traffico, caratteristiche che possono risultare critiche in caso di eventi imprevisti.

Controllo reattivo. Una regola locale osserva lo stato corrente dell'intersezione (tipicamente il numero di veicoli in coda per corsia) e sceglie l'azione che ottimizza un criterio istantaneo, senza affidarsi a un piano prestabilito. Questa regola può essere scritta a mano, controllo *euristico*, oppure appresa dall'esperienza tramite reinforcement learning, controllo *appreso*: quest'ultimo è il paradigma adottato da

questo lavoro (§2.2). All'interno del controllo euristico, Shams et al. (2018) distinguono ulteriormente il grado di adattività: il controllo *attuato* più semplice si limita a prolungare o abbreviare le fasi di un piano di base ancora sostanzialmente predefinito (estendere una fase finché un sensore rileva veicoli sulla corsia servita, ad esempio), mentre un controllo pienamente *adattivo* ricalcola la struttura stessa del piano, non solo la sua durata, in risposta al traffico osservato. Piattaforme come SCATS e SCOOT si collocano in questa seconda fascia, coordinando intere aree urbane aggiustando periodicamente i piani semaforici sulla base del traffico osservato, pur restando ancorate a una struttura di piano pre-esistente (Wei et al., 2020).

MAX-PRESSURE (Varaiya, 2013) rappresenta invece un sistema di controllo reattivo che, sotto l'ipotesi idealizzata di corsie a capacità infinita, stabilizza le code della rete per qualunque livello di domanda che la rete stessa sia in grado di sostenere, senza alcuna necessità di training. Questo risultato, pur non potendo essere traslato integralmente in scenari reali dove non è rara la possibilità di una completa saturazione di una via, si fonda solamente su dati raccolti a ogni istante decisionale nelle corsie adiacenti all'incrocio e su un principio di base: selezionare a ogni istante la fase a pressione massima. Data una fase semaforica ϕ , la sua pressione è definita come la somma, su tutti i movimenti abilitati da ϕ , della differenza tra il numero di veicoli in coda sulla corsia di ingresso e quelli sulla corsia di uscita, contati sull'intera corsia:

$$P(\phi) = \sum_{(\ell_{\text{in}}, \ell_{\text{out}}) \in \text{movimenti}(\phi)} \left(n(\ell_{\text{in}}) - n(\ell_{\text{out}}) \right), \quad (2.12)$$

dove $n(\ell)$ indica il numero di veicoli sulla corsia ℓ . Questa è la forma semplificata della pressione usata in questo lavoro e in gran parte della letteratura successiva, che assegna un solo movimento a ogni coppia di corsie e non richiede quindi rapporti di svolta (la frazione di veicoli di una corsia diretta verso ciascuna corsia di uscita raggiungibile), necessari solo quando una stessa corsia in ingresso alimenta più movimenti distinti. A ogni step di decisione, l'algoritmo calcola la pressione di tutte le fasi e seleziona quella a pressione massima, $\phi^* = \operatorname{argmax}_{\phi} P(\phi)$, senza alcun piano né ciclo predefinito, né alcun vincolo sulle azioni consecutive: la stessa fase può in linea di principio essere selezionata un numero indefinito di volte. Uno svantaggio è la necessità di sensoristica capace di monitorare le corsie per la loro interezza. Il problema principale, tuttavia, è l'assenza di garanzie di equità: in presenza di un forte afflusso di traffico su un certo insieme di corsie, la pressione maggiore può trovarsi sempre sulle fasi che coinvolgono quelle vie, con conseguenti tempi di attesa eccessivi per i veicoli sulle vie meno trafficate, un aspetto di equità che questo lavoro esamina esplicitamente.

Fixed-Time e Max-Pressure sono le due baseline sperimentali classiche a cui questo lavoro confronta i propri modelli (Capitolo 4), usate qui esattamente nella forma appena descritta. Pur fondandosi su paradigmi opposti, statico l'uno e adattivo l'altro, entrambe non richiedono alcuna fase di addestramento, il che le rende candidati naturali per il confronto con i modelli di deep reinforcement learning discussi nel resto di questo lavoro.

2.5.3 Simulazione

Valutare una politica di controllo semaforico su una rete reale non è praticabile durante lo sviluppo, in quanto cambiare i tempi di un semaforo cittadino migliaia di volte per tentativi, come richiederebbe l'addestramento di un agente RL, non è ovviamente proponibile. Serve quindi un simulatore che riproduca la dinamica del traffico con un dettaglio sufficiente, pur mantenendo un costo computazionale accettabile per l'addestramento di un centinaio di episodi (Capitolo 3).

Un simulatore di traffico microscopico rappresenta ogni veicolo individualmente, con una propria posizione, velocità e traiettoria, aggiornate a ogni istante da un modello che regola la velocità di un veicolo in base al comportamento di quello che lo precede mantenendo una distanza di sicurezza. È il livello di dettaglio presente in questo lavoro, sufficientemente fedele da catturare l'effetto di una singola decisione semaforica su una coda specifica.

CITYFLOW (Zhang et al., 2019) è il simulatore microscopico scelto in questo lavoro. Rispetto ad alternative più diffuse come SUMO (Lopez et al., 2018), CITYFLOW semplifica ulteriormente il modello che regola la dinamica dei veicoli per privilegiare la velocità di simulazione, un compromesso adatto quando l'obiettivo primario è addestrare un agente su un numero elevato di episodi piuttosto che riprodurre con precisione millimetrica la dinamica di un incrocio specifico.

2.5.4 Controllo semaforico appreso: stato dell'arte

Il controllo appreso sostituisce la regola scritta a mano del controllo reattivo con una politica addestrata sull'esperienza, ma non elimina il problema del coordinamento tra intersezioni: lo sposta, dalla progettazione manuale di un piano coordinato alla scelta di come far condividere informazione agli agenti durante l'apprendimento. La letteratura sul reinforcement learning applicato al TSC si può raggruppare, non esaustivamente, per come affronta questa scelta.

RL su singola intersezione. I primi lavori applicano il RL a un solo incrocio alla volta, spesso come banco di prova per verificare che un agente possa competere con le regole reattive prima ancora di affrontare il problema del coordinamento. AttendLight (Oroojlooy et al., 2020) ne è un esempio recente: addestra un singolo modello su un insieme eterogeneo di intersezioni indipendenti, e ne verifica le prestazioni su intersezioni mai viste, ma dichiara esplicitamente che l'estensione a una rete di più intersezioni coordinate resta lavoro futuro.

Multi-agente indipendente. Un secondo filone affronta direttamente una rete di intersezioni, ma senza alcuno scambio esplicito di informazione tra agenti vicini: ciascuna intersezione apprende e decide sulla base del proprio solo stato locale, e un'eventuale coordinazione emerge indirettamente dal fatto che le azioni di un agente alterano il traffico osservato dagli altri. El-Tantawy et al. (2013) propongono un primo schema di questo tipo su scala urbana reale, mentre Chu et al. (2019) confrontano esplicitamente varianti indipendenti e cooperative dello stesso algoritmo su reti di grandi dimensioni. Il vantaggio di questo filone è la scalabilità, ogni agente resta un problema di RL a singolo agente; lo svantaggio è che nessuna azione tiene conto esplicitamente dello stato dei vicini nel momento della decisione.

Coordinato tramite comunicazione o reti a grafo. Un terzo filone fa scambiare esplicitamente informazione tra intersezioni vicine durante la decisione stessa, non solo indirettamente attraverso l'ambiente. Zhang et al. (2025) modellano esplicitamente le dipendenze tra agenti vicini in questo schema. CoLight (Wei et al., 2019) usa un meccanismo di *graph attention* tra intersezioni adiacenti per decidere quanto pesare il contributo di ciascun vicino, con pesi condivisi da tutti i nodi della rete. MetaSTGAT (Wang et al., 2022), il modello di riferimento di questo lavoro, condivide con CoLight il meccanismo di comunicazione via *graph attention*, ma vi combina un modulo di meta-learning (§2.4): invece di condividere gli stessi pesi tra tutte le intersezioni della rete, questo modulo genera dinamicamente i pesi di uno strato di attenzione e di uno strato ricorrente a partire da caratteristiche locali dell'intersezione, così che nodi diversi possano essere serviti da trasformazioni diverse pur restando un unico modello addestrato end-to-end. Gli autori originali riportano una riduzione del travel time medio rispetto a CoLight, un metodo di riferimento tra i più citati per il TSC.

Equità. Indipendentemente dallo schema di coordinamento adottato, alcuni lavori affrontano esplicitamente il problema dell'equità tra direzioni di traffico, penalizzando le fasi responsabili di attese eccessive invece di ottimizzare solo una metrica

aggregata di rete. FELight (Xiao et al., 2025) ne è un esempio: introduce una metrica di equità basata sui tempi di attesa dei veicoli e penalizza le fasi già identificate come responsabili di un ritardo superiore a una soglia prestabilita. Un tratto comune a questa famiglia di approcci è intervenire in modo selettivo, solo quando un’attesa supera una soglia o su una fase individuata in anticipo come problematica, invece che in ogni istante decisionale e per ogni intersezione.

Generalizzazione. La prassi più diffusa nel controllo semaforico appreso è addestrare e valutare un modello sulla stessa rete stradale, al più ripetendo l’intero esperimento, addestramento incluso, su una rete diversa, non trasferendo un singolo modello già addestrato a una rete che non ha mai visto (Wei et al., 2020). CoLight (Wei et al., 2019), per esempio, conduce cinque esperimenti indipendenti, due su reti sintetiche e tre su reti reali, ciascuno con il proprio addestramento; MetaSTGAT (Wang et al., 2022) segue lo stesso schema su una griglia sintetica e due dataset reali. Quando in questi lavori compare il termine *generalization*, indica quasi sempre la tenuta a diverse intensità di traffico sulla stessa rete, non a una topologia strutturalmente diversa (Wang et al., 2026; Sun et al., 2026), come reso esplicito da Zhang et al. nella motivazione di GeneraLight (Zhang et al., 2020). Le eccezioni restano circoscritte a un’unica dimensione di variazione: AttendLight generalizza tra intersezioni singole ma non a una rete coordinata, come già notato sopra; Advanced-XLight (Zhang et al., 2022) misura un rapporto di trasferimento tra un modello addestrato su un dataset reale e valutato direttamente su un altro, senza ulteriore addestramento, ma sempre tra reti di scala paragonabile; lo stesso GeneraLight generalizza al variare del flusso di traffico, non della topologia della rete.

2.6 Sintesi

Il controllo semaforico su rete è un problema di coordinamento distribuito che le regole scritte a mano affrontano con euristiche fisse, poco adattabili a condizioni non previste in fase di progettazione, e che il controllo appreso affronta invece con una politica addestrata sull’esperienza simulata. All’interno del controllo appreso, la letteratura ha esplorato schemi di coordinamento via via più espliciti, dalla singola intersezione al multi-agente indipendente fino alla comunicazione tramite reti neurali a grafo, un asse su cui MetaSTGAT si colloca con un meccanismo di meta-learning che permette a intersezioni di topologia diversa di condividere comunque un unico modello.

Su due assi, però, la letteratura resta parziale. I meccanismi di equità esistenti intervengono in modo selettivo, solo oltre una soglia o su fasi individuate in anticipo, invece che come obiettivo esplicito valutato a ogni istante e per ogni intersezione. La generalizzazione, quando misurata, riguarda quasi sempre l'intensità del traffico sulla rete di addestramento, raramente una topologia, una spaziatura stradale o un profilo di traffico strutturalmente diversi da quelli visti in addestramento, e le eccezioni che esistono restano confinate a un'unica dimensione di variazione alla volta. I capitoli seguenti descrivono come questo lavoro affronta congiuntamente questi due assi, usando MetaSTGAT come architettura di partenza.